



UNIVERSIDADE FEDERAL DE LAVRAS - UFLA
CAMPUS SÃO SEBASTIÃO DO PARAÍSO
Bacharelado Interdisciplinar em Inovação, Ciência e Tecnologia

RELATÓRIO TÉCNICO: PROJETO 02
Algoritmos de Percurso em Grafos e Identificação de Componentes Conexas

Milena Duarte Costa

Professor: Dr. Alysson Naves

São Sebastião do Paraíso - MG
Maio de 2026

Sumário

1	Introdução	2
2	Metodologia de Implementação	2
2.1	Representação do Grafo	2
2.2	Busca em Largura (BFS)	2
2.3	Busca em Profundidade (DFS)	2
3	Análise e Resultados	3
3.1	Grafo de Teste	3
3.2	Resultados dos Percursos	3
3.3	Identificação de Componentes Conexas	3
4	Análise Conceitual	4
4.1	Qual a diferença conceitual entre BFS e DFS?	4
4.2	Por que BFS encontra menor caminho em grafos não ponderados?	4
4.3	O que é uma componente conexa?	4
5	Conclusão	4

1 Introdução

Este relatório apresenta o desenvolvimento e a análise do Projeto 02 da disciplina GCT153 - Estruturas de Dados II. O objetivo central foi a implementação de uma estrutura de dados de Grafo Não Direcionado utilizando Lista de Adjacência, bem como a aplicação de algoritmos clássicos de busca: Busca em Largura (BFS) e Busca em Profundidade (DFS). Adicionalmente, foi desenvolvido um método para identificação e contagem de Componentes Conexas do grafo.

2 Metodologia de Implementação

2.1 Representação do Grafo

A estrutura foi implementada de forma modular através de uma classe em Python. Optou-se pelo uso de um dicionário de listas para representar a **Lista de Adjacência**, onde cada chave representa um vértice e sua respectiva lista contém os vértices vizinhos. Esta escolha justifica-se pela eficiência de memória em grafos esparsos e pela facilidade de iteração sobre vizinhos.

2.2 Busca em Largura (BFS)

A BFS foi implementada utilizando uma **fila** (através da estrutura `collections.deque`). O algoritmo inicia em um vértice raiz, marca-o como visitado e explora todos os seus vizinhos imediatos antes de avançar para os vértices de nível dois. A ordem de visita foi registrada para validação dos resultados.

2.3 Busca em Profundidade (DFS)

A DFS foi implementada de forma **recursiva**, permitindo que o algoritmo explore cada ramo até o limite máximo antes de realizar o *backtracking*. Assim como na BFS, utilizou-se um conjunto de controle para evitar ciclos e registrar a ordem de visita correta.

3 Análise e Resultados

3.1 Grafo de Teste

Para a validação, utilizou-se o grafo obrigatório composto por 10 vértices (0 a 9) e as seguintes arestas: (0,1), (1,2), (2,3), (4,5), (6,7), (7,8) e (8,6). O vértice 9 foi mantido como um nó isolado.

3.2 Resultados dos Percursos

Após a execução do código, foram obtidos os seguintes resultados de visitação a partir do vértice inicial 0:

- **Ordem de Visita BFS:** 0, 1, 2, 3.
- **Ordem de Visita DFS:** 0, 1, 2, 3.

3.3 Identificação de Componentes Conexas

O algoritmo percorreu todos os vértices não visitados e aplicou as buscas para delimitar cada subgrafo. Foram identificadas as seguintes componentes:

Tabela 1: Componentes Conexas Identificadas

Identificador	Vértices Pertencentes
Componente 1	0, 1, 2, 3
Componente 2	4, 5
Componente 3	6, 7, 8
Componente 4	9

Total de componentes encontradas: 4.

4 Análise Conceitual

4.1 Qual a diferença conceitual entre BFS e DFS?

Conceitualmente, a **BFS (Busca em Largura)** foca na proximidade, visitando todos os vizinhos de um nível antes de passar para o próximo nível de profundidade. Já a **DFS (Busca em Profundidade)** foca na exploração de caminhos, descendo o mais fundo possível em um ramo antes de retornar para explorar alternativas. A BFS utiliza uma fila (FIFO), enquanto a DFS utiliza uma pilha (LIFO), seja ela explícita ou via pilha de recursão.

4.2 Por que BFS encontra menor caminho em grafos não ponderados?

A BFS encontra o menor caminho porque ela explora o grafo em ondas de distância uniforme. Como ela visita todos os nós a uma distância k antes de qualquer nó a uma distância $k + 1$, a primeira vez que ela atinge um vértice de destino, garante-se que o caminho percorrido é aquele com o menor número de arestas possível a partir da origem.

4.3 O que é uma componente conexa?

Uma componente conexa é um subgrafo maximal em que existe pelo menos um caminho entre qualquer par de vértices pertencentes a esse subgrafo. Em termos práticos, em um grafo não direcionado, uma componente conexa representa um "conjunto de ilhas" ou partes do grafo que estão isoladas entre si.

5 Conclusão

O projeto permitiu consolidar os conceitos de representação de grafos e a mecânica dos algoritmos de busca. A implementação em Python mostrou-se eficiente para o problema proposto, e os resultados obtidos coincidiram com o esperado para o grafo de teste sugerido pelo docente. A identificação de componentes conexas demonstrou ser uma ferramenta poderosa para entender a conectividade global de sistemas estruturados em grafos.